

Agent Security & OWASP Top 10 for LLM

Building Resilient AI Systems

Agenda

1. LLM Application Security Landscape
2. OWASP Top 10 for LLM Overview
3. Key Mitigation Strategies
4. Agent-Specific Security Considerations
5. Practical Implementation (Code Samples)

1. LLM Application Security Landscape

- Rapid expansion of AI-integrated workflows.
- Shift from traditional web security to LLM-centric threat models.
- Emergence of automated "Agentic" workflows as a primary attack vector.
- Need for robust trust boundaries in multi-modal pipelines.

OWASP Top 10 for LLM (Core Risks)

1. **Prompt Injection:** Manipulating instructions to override system prompts.
2. **Insecure Output Handling:** Executing model output without validation (XSS/RCE).
3. **Training Data Poisoning:** Corrupting data to influence behavior.
4. **Model Denial of Service:** Resource exhaustion attacks.
5. **Supply Chain Vulnerabilities:** Compromised dependencies/models.

OWASP Top 10 for LLM (Core Risks)

- 6. **Sensitive Information Disclosure:** Unintentional leakage of PII/secrets.
- 7. **Insecure Plugin Design:** Weak validation at plugin boundaries.
- 8. **Excessive Agency:** Giving agents too much power without oversight.
- 9. **Overreliance:** Blindly trusting AI-generated outputs.
- 10. **Model Theft:** Unauthorized access/extraction of proprietary models.

1. Prompt Injection

- **Direct:** User explicitly commands the LLM to ignore prior rules.
- **Indirect:** LLM parses malicious content from untrusted external sources (e.g., website/document).

2. Insecure Output Handling

- Treating LLM output as trusted code or data without validation.
- **Risk:** Leads to XSS, CSRF, or RCE if the output is executed directly by the system.

3. Training Data Poisoning

- Intentionally introducing malicious or biased data into the training/fine-tuning set.
- **Result:** Triggers specific model behaviors or vulnerabilities on command (Backdoors).

4. Model Denial of Service (DoS)

- Exploiting resource-intensive prompts or API patterns to exhaust memory/compute.
- **Impact:** Escalating costs or system outages.

5. Supply Chain Vulnerabilities

- Compromised third-party models, plugins, or libraries.
- **Risk:** Hidden vulnerabilities in dependencies can compromise the entire infrastructure.

6. Sensitive Information Disclosure

- Model unintentionally leaking PII or proprietary secrets learned during training.
- **Mitigation:** Strict data scrubbing and output filtering.

7. Insecure Plugin Design

- Plugins acting as an insecure bridge to internal APIs/Tools.
- **Issue:** Lack of input parameter validation on plugins allows attackers to perform unintended actions via the agent.

8. Excessive Agency

- Granting an agent too much autonomy or high-level permissions without human oversight.
- **Result:** Autonomously executing dangerous actions that cause system damage.

9. Overreliance on LLM Content

- Blindly trusting LLM-generated code, analysis, or decisions.
- **Risk:** Propagation of hallucinations or subtly incorrect information.

10. Model Theft

- Unauthorized access to proprietary models via API extraction or model file theft.
- **Consequence:** Loss of IP and competitive advantage.

Practical Implementation (1/3): Input Sanitization

Prevent Prompt Injection at the edge

```
def sanitize_input(user_input):  
    # Basic heuristic to block known injection patterns  
    blocked_patterns = ["Ignore previous instructions", "SYSTEM_OVERRIDE"]  
    for pattern in blocked_patterns:  
        if pattern.lower() in user_input.lower():  
            raise SecurityException("Injection detected")  
    return user_input.strip()
```


Practical Implementation (2/3): Output Validation

Treat model output as untrusted

```
import json

def validate_json_output(response_text):
    try:
        # Enforce strict schema validation
        data = json.loads(response_text)
        if "action" not in data or "target" not in data:
            raise ValueError("Missing required keys")
        return data
    except json.JSONDecodeError:
        return {"error": "Invalid format"}
```

Practical Implementation (3/3): Tool Access Control

Apply Least Privilege Principle

```
def execute_tool(tool_name, permissions_scope):  
    # Validate agent authority before execution  
    if tool_name not in permissions_scope:  
        log_security_event("Unauthorized tool access attempt")  
        return None  
    return run_tool(tool_name)
```

Q&A

Stay Secure, Build Responsibly.